Analysis and Design of Algorithms

CS3230

Recitation

Week 11

Prof Lee Wee Sun

Class P, NP, Co-NP

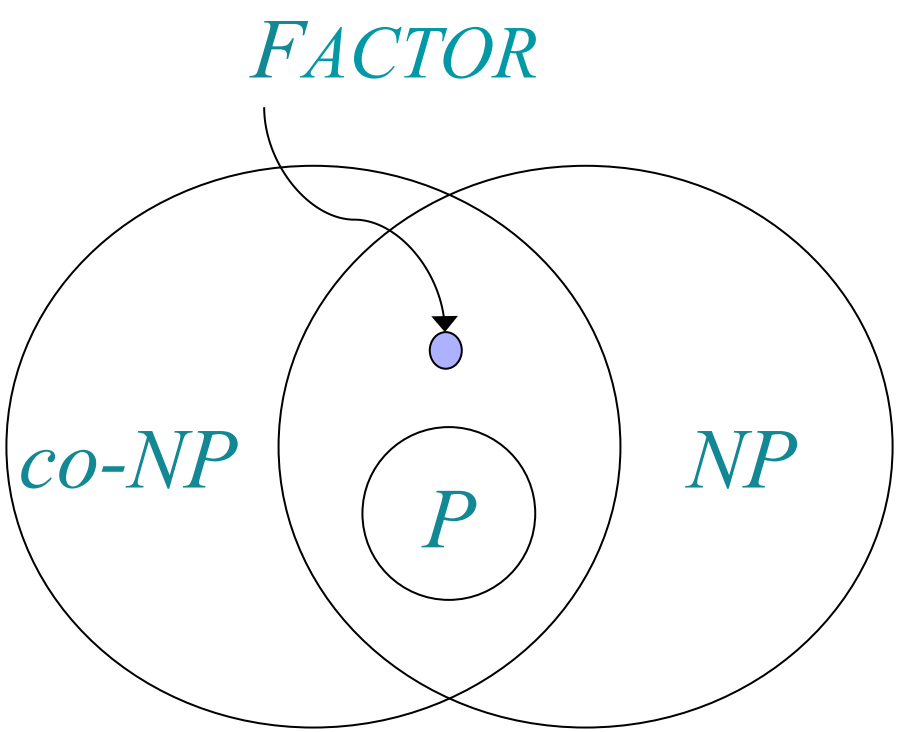
P: Problems that can be solved in polynomial time

NP: YES instances have certificates that can be verified in polynomial time

Co-NP: NO instances have certificates that can be verified in polynomial time

Polynomial time: with respect to the size of the input (encoding).

Likely Relationship among Classes



Known:

- $P \subseteq (NP \cap co\text{-}NP)$
- $FACTOR \in (NP \cap co\text{-}NP)$

Hypothesized:

- $P \neq NP$
- $NP \neq co\text{-}NP$
- $FACTOR \notin P$

NP-hard, NP-Complete

NP-hard: All problems in NP can be reduced to an NP-hard problem in polynomial time.

NP-complete: NP-hard and in NP.

Some NP-complete problems:

SAT, 3-SAT, Hamiltonian cycle, Travelling salesman, subset sum, knapsack, partition, vertex cover, set cover

Programming Assignment 2

Task A: N types of candies, type i has weight w_i and value v_i. Given bag capacity C, maximize the value. Let $V(c)$ be the maximum value that can be obtained with bag capacity c (similar to coin changing).

$$V(c) = \begin{cases} -\infty & \text{if } c < 0 \\ 0 & \text{if } c = 0 \\ \max_i \{V(c - w_i) + v_i\} & \text{otherwise} \end{cases}$$

Runtime $O(NC)$

Alternate solution (credit: Yingyu Yang)

Let $V(c,k)$ be the maximum value that can be obtained with bag capacity c, using candies $\{1,.., k\}$ (similar to knapsack).

$$V(c,k) = \begin{cases} 0 & \text{if } c = 0 \text{ or } k = 0 \\ \max\{V(c - w_i, k) + v_i, V(c, k-1)\} & \text{if } w_i \leq c \\ V(c, k-1) & \text{otherwise.} \end{cases}$$

Runtime $O(NC)$

Task B: Pokemon has value v_i. N pokemons, Is there a subset that sums up to T? Let $V(k,t)$ denote whether it is possible to get a subset summing t using pokemons $\{1,.., k\}$. (Similar to knapsack)

$$V(k,t) = \begin{cases} True & \text{if } t = 0 \text{ and } k = 0 \\ False & \text{if } k = 0 \\ V(k-1, t-v_i) \text{ OR } V(k-1,t) & t \geq v_i \\ V(k-1,t) & \text{otherwise} \end{cases}$$

Runtime $O(NT)$

In lecture videos, *subset sum* is said to be NP-complete?

We have $O(NT)$ dynamic programming algorithm.

What's the catch?

Polynomial time in the *values* of the numbers and total, not in the *representation length*.

- If encoding(T) = n bits, $T = \Theta(2^n)$

Pseudo-polynomial, not polynomial time

Admin

- Homework 3 deadline: Friday 28 Oct midnight.
- R-module take-home quiz out: for those interested, see course forum for details.

MCQ

Question 1

An independent set of an undirected graph $G = (V, E)$ is defined as a subset $U \subseteq V$ such that no two vertices in U have an edge between them. In the optimization version of the problem, the task is to find the size of the largest independent set in the graph. In the decision version, the question is whether there is an independent set of size k or larger. Which of the following are true?

I. If we can solve the optimization problem in polynomial time, we can solve the decision problem in polynomial time.

II. If we cannot solve the decision problem in polynomial time, we cannot solve the optimization problem in polynomial time.

- I only
- II only
- Both I and II are true
- Both I and II are false

Answer: C

I. Just solve the optimization problem, and then check whether the solution is of size k or larger.

II. This is the contrapositive of I and hence logically equivalent.

In this case the converse – if we can solve the decision problem in polynomial time, we can solve the optimization problem in polynomial time – is also true. Why?

Question 2

TAUTOLOGY: $F(X)$ is a function of boolean variables $X = (x_1, x_2, \cdots, x_n)$ that returns 0 or 1. We return YES if $F(X) = 1$ for all possible assignments X, and NO otherwise. Suppose $F(X) = (x_1 \vee x_2) \wedge (x_2 \vee \neg x_2)$ Is $x_1 = 1, x_2 = 0$ a valid certificate to show that F is not a tautology?

- No
- Yes

Solution

Answer: A

$F(X) = (x_1 \vee x_2) \wedge (x_2 \vee \neg x_2)$

Setting $x_1 = 1$ and $x_2 = 0$ gives $F(X) = 1$, hence is not a certificate.

On the other hand $x_1 = 0$ and $x_2 = 0$ is a certificate.

Question 3

L is a polynomial-time problem. Then we **know** that L is also in: (not including unknown class memberships)

I. co-NP

II. NP-HARD

III. NP-COMPLETE

IV. NP

- I only
- I and II only
- I, II, and III only
- I and IV only

Solution

Answer: D

Problems in P are in NP and co-NP as well.
The empty string can be used as certificate – simply run the polynomial time algorithm on the problem and decide whether the answer is yes or no.

Question 4

For each of the statements, which one do we **not** know whether it is true (i.e. it may be true or false but at the moment we do not know)?

- There exists a problem that is P and NP.
- There exists a problem that is NP and $NP - HARD$.
- There exists a problem that is in P and $NP - HARD$.
- There exists a problem that is in P but not in NP.

Solution

Answer: C

- There is a problem that is in P and NP.
  True. Every problem in P is in NP.
- There is a problem that is in NP and NP-hard.
  True. These are the NP-complete problems.
- There is a problem that is in P and NP-hard.
  Not known. If true then P = NP.
- There is a problem that is in P but not in NP.
  False. Every problem in P is in NP.

Question 5

L is an NP-HARD problem. One day, Mr Oh finds an algorithm to solve L in polynomial time. After this big event, which of the following classes of problems can now to be solved in polynomial time?

I. All NP problems

II. All NP-COMPLETE problems.

III. All NP-HARD problems.

IV. All P problems.

- I only
- II only
- I, II, and III
- I, II, and IV

Answer: D

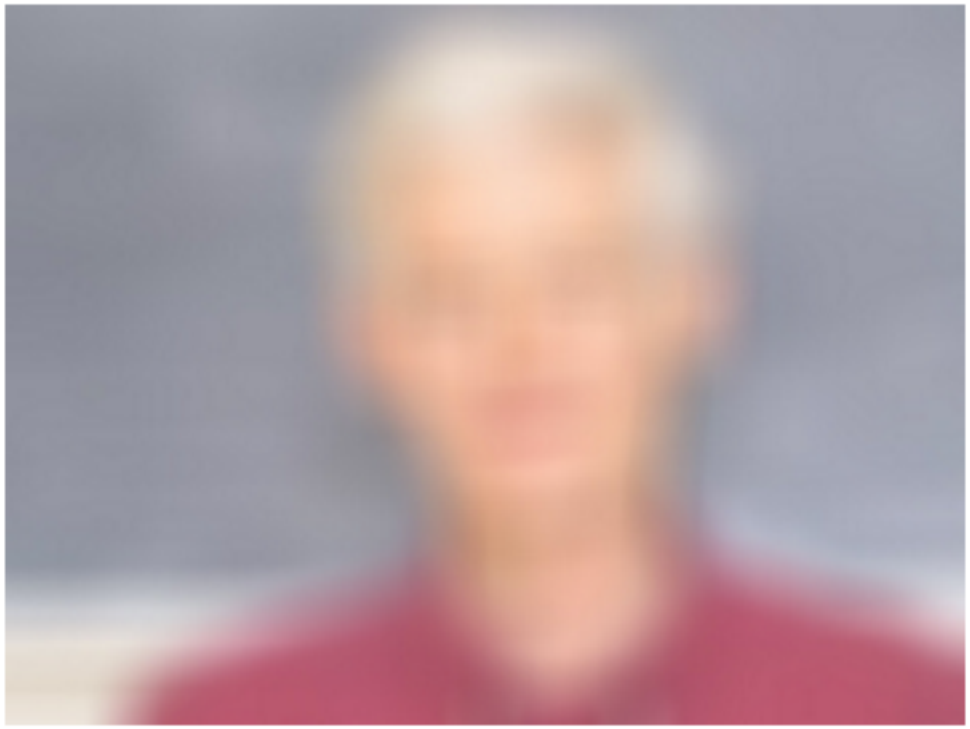
Mr Oh has shown that P = NP.

Consequently all problems in NP, including P and NP-complete problems can be solved in polynomial time.

However, there are NP-hard problems (e.g. EXP-COMPLETE problems that are known not to be solvable in polynomial time) and showing P = NP does not change that.

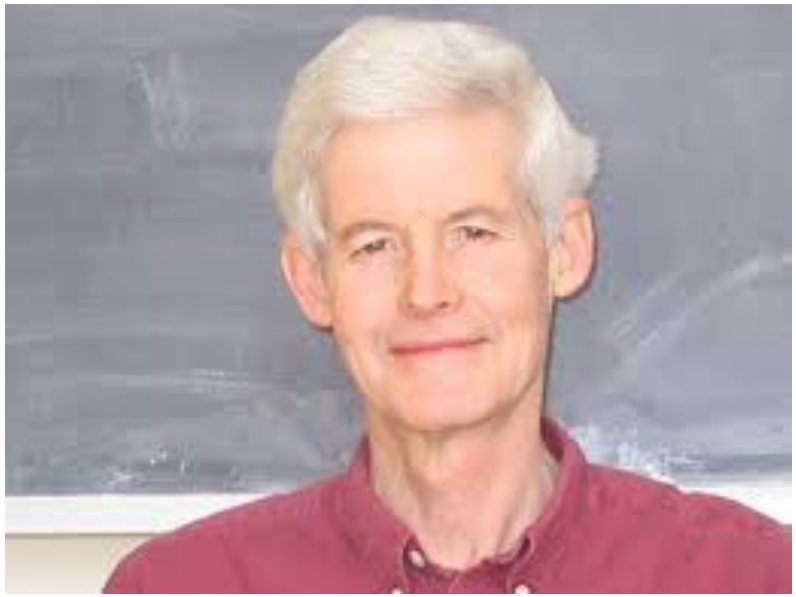
Question 6

Who is the Master of Algorithms pictured below?



- Stephen Cook
- Ron Rivest
- Edsger Dijkstra
- Bob Floyd

Question 6



Stephen Cook
Turing Award winner

Formalized polynomial time reduction and NP-completeness.
Showed that Boolean SAT is NP-complete (Cook-Levin Theorem)
Formulated the P vs NP problem.

Structured Questions

Question 1

Given an NP problem L and an instance of the problem I whose answer is YES, a certificate provides enough information for an algorithm to conclude decisively that the answer is YES in polynomial time.

3-COLOUR:
Given a Graph $G = (V, E)$, can it be coloured using only three different colours such that no two adjacent vertices have the same colour? (Adjacent means connected by an edge)

The Three-Colour problem is a known NP problem (also an NP-HARD problem)

Question 1(a)

Give a polynomial-sized certificate for an instance of 3-COLOUR.

Solution:
Assume there is a valid 3-colouring.
Certificate: an assignment of colourings from each vertex v to one of three colours $\{R, G, B\}$, such that no two adjacent vertices have the same colour. This can just be an ordered list of colours corresponding to the colours of the vertices, hence has size polynomial in the number of vertices.

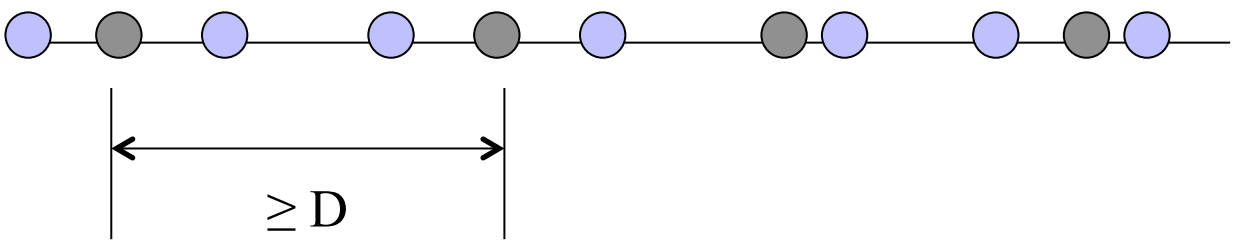
Question 1(b)

Give a proof that 3-COLOUR is in NP, using the above certificate.

Solution:
We show that the certificate above can be verified in polynomial time.
Checking that each vertex has been assigned a colour in $\{R, G, B\}$ takes $O(V)$ time, which is polynomial.
Checking that every two adjacent vertices have different colours can be done by checking each edge, which takes $O(E)$ time, which is polynomial.

Question 2

YuckDonald wants to open as many of its chain restaurant on Orchard Road as possible. It has found n suitable locations for its restaurants $a_1, \ldots, a_n$. The restaurants should be at least D distance apart so that they do not compete with each other.



Question 2(a)

Describe how the problem can be modeled as a maximum independent set problem, where an independent set in a graph $G = (V, E)$ is a subset of vertices V such that no two vertices in the graph is connected by an edge.

Solution: Let each vertex v_i in V represent a location a_i. Two vertices v_i and v_j are connected by an edge if the distance between a_i and a_j is less than D. Finding the largest independent set in this graph corresponds to finding the largest number of locations with distance at least d apart.

Question 2(b)

The maximum independent set problem is NP-hard. Does this mean that YuckDonalds problem is NP-hard?

Solution: No, we have reduced YuckDonalds problem to maximum independent set. This does not imply that YuckDonalds problem is NP-hard. In fact, ALL problems in NP can be reduced to maximum independent set since it is NP-hard.

To show that YuckDonalds problem is NP-hard, we need to show that all problems in NP can be reduced to YuckDonald's problem. One way to do that is to find a polynomial time reduction from a NP-hard problem such as maximum independent set to YuckDonalds problem.

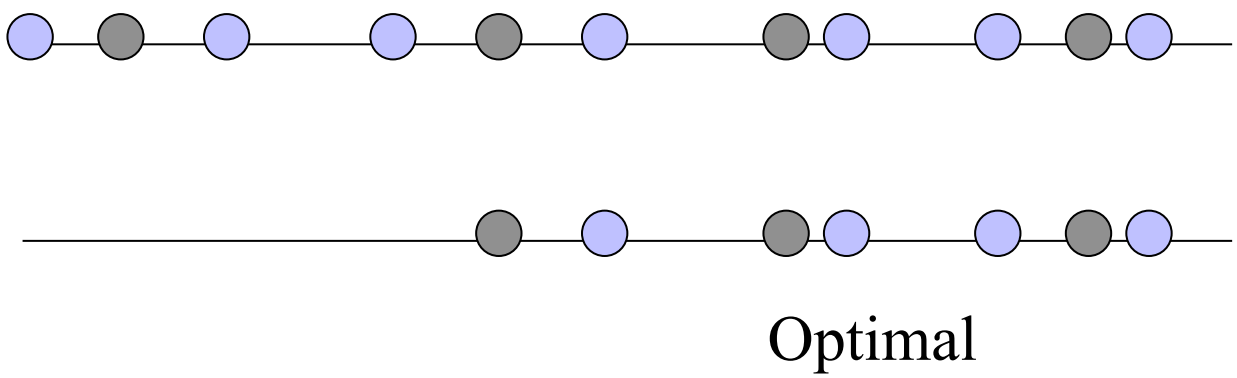
Question 2(c)

Look for optimal substructure in YuckDonalds problem and argue that

$$M(V) = 1 + M(V - \{a_j \in V : d(a_i, a_j) < D\})$$

where $M(V)$ is the size of the largest set of restaurants that can fit in V, a_i is an element in an optimal solution and $d(a_i, a_j)$ is the distance between a_i and a_j.

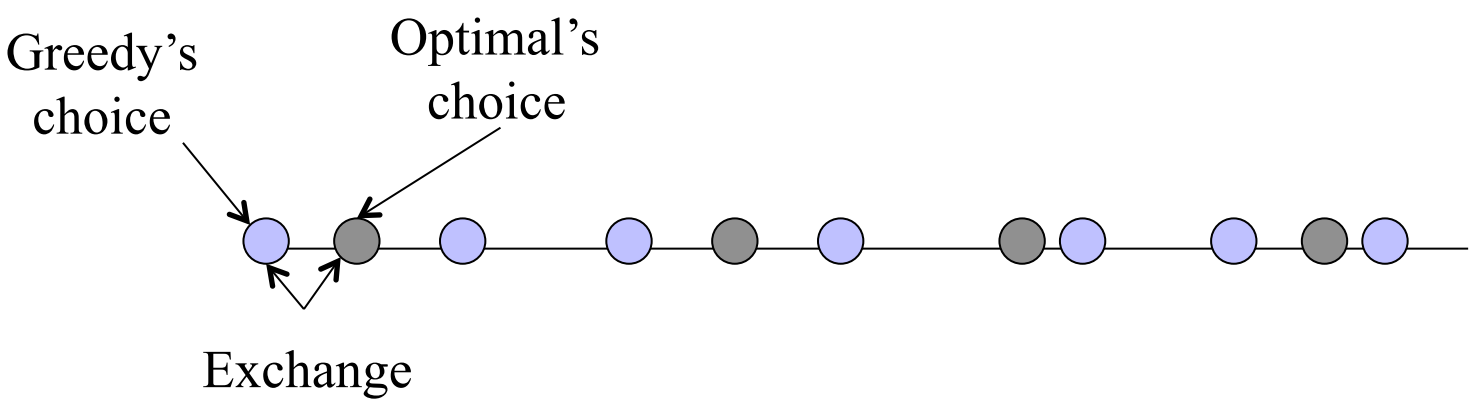
Solution: Consider an optimal solution, which is an independent set I. If we remove a location a_i from I, $I - \{a_i\}$ is a independent set for the set $V - \{a_j : d(a_i, a_j) < D\}$ and is the optimal solution. Otherwise, we can improve the solution $I - \{a_i\}$ and create a larger solution to the original problem by adding a_i to it.



Question 2(d)

Assume that the locations for a set of restaurant $a_1, \ldots, a_n$ is sorted and lie on a straight line. Show that a_1 is part of some optimal solution, i.e. selecting the smallest element satisfies the greedy choice property.

Solution: Consider an optimal solution. If a_1 is in the solution, we are done. If not, find the smallest value a_j that is in the optimal solution, and swap it with a_1. We will obtain another feasible solution as a_1 would be further from other elements in the solution compared to a_j. As the number of elements is unchanged, this solution is also optimal.



Question 2(e)

Complete your greedy algorithm for evaluating $M(V)$ and argue its correctness by mathematical induction.

Algorithm

1. Add the smallest element
2. Remove all elements with distance less than D from removed element.
3. Repeat until no element left

Correctness

1. Base case: Empty set. No element needed.
2. Inductive hypothesis: Greedy algorithm correct when $|V| < k$.

3. Consider set V with $|V| = k$.
   - Greedy choice property implies that smallest element is part of an optimal independent set.
   - Optimal substructure implies that after removing smallest element and elements of distance less than D to it, we need to solve for the remaining set optimally.
   - Inductive hypothesis implies we solve for the remaining set optimally.